KIROVAULT

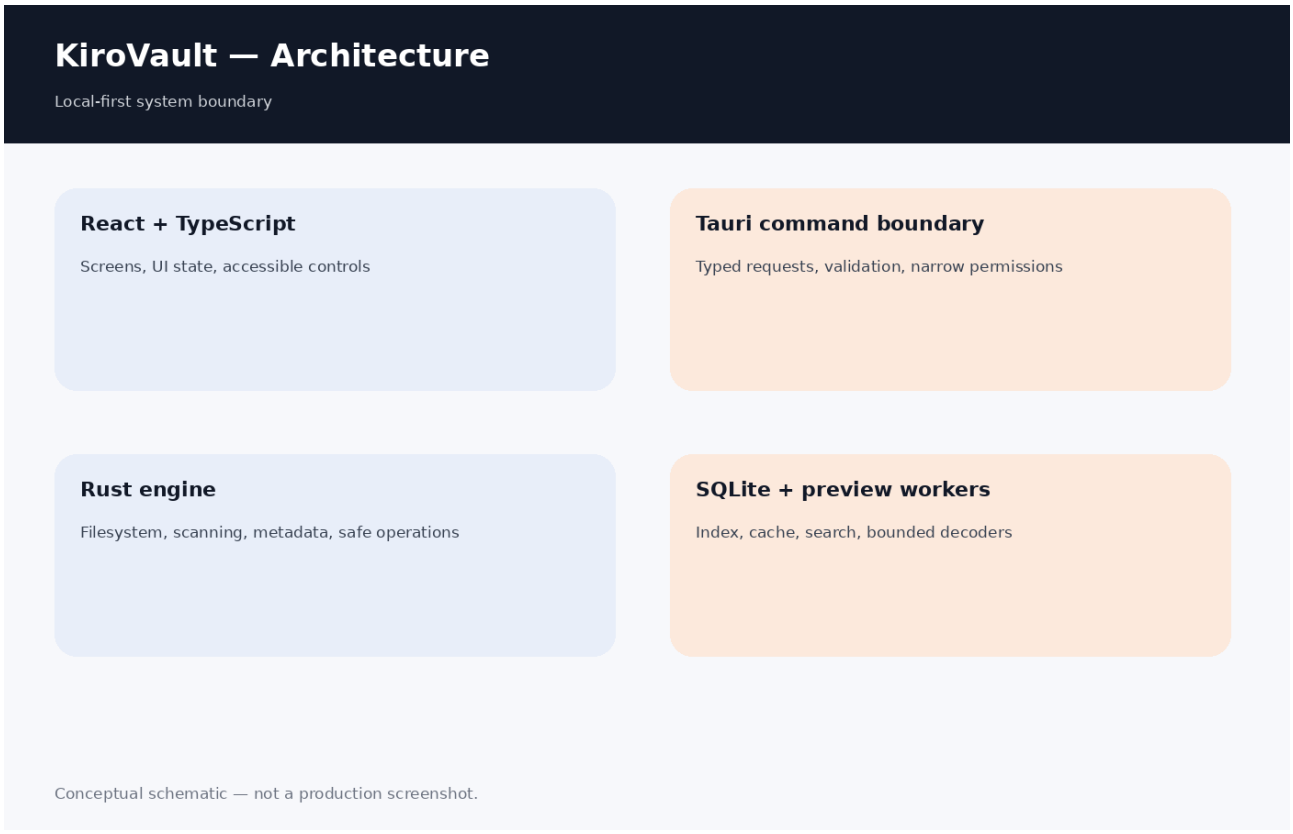
Complete Technical Architecture

A Windows-first, local-first architecture based on Tauri 2, React, TypeScript, Rust and SQLite, as proposed in the master blueprint.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Architecture reference image



Contents and implementation guidance

System layers

• React + TypeScript: screens, accessible UI state, selection, grid/list/timeline and preview inspector.

• Tauri 2: desktop lifecycle, packaging and narrow frontend-to-native command boundary.

• Rust: filesystem operations, validation, scanning, metadata, thumbnail workers and typed results.

• SQLite: local index, favorites, tags, settings, collections, scan checkpoints and optional FTS5.

• Preview workers: image/PDF/audio/video-specific adapters, bounded queues and local cache.

Command boundary

• Expose task-specific commands rather than unrestricted filesystem access.

• Validate canonical paths, permissions, scope, symlink/junction behavior and collision policies in Rust.

• Return structured success, conflict, partial-failure and error results.

• Keep frontend service wrappers typed; never treat a UI path string as authorization.

Data flow

• UI requests directory listing → Rust validates access → returns metadata → UI renders visible rows.

• UI requests visible thumbnails → cache lookup → bounded worker generates preview → cache stores result → UI updates.

• Index scan runs in background with progress, pause/resume/cancel; browsing remains responsive.

• Original files stay in their actual locations; index/cache are rebuildable.

Reliability boundaries

• Separate listing from thumbnail generation; never block folder open on all previews.

• Use timeouts and per-file resource limits for preview workers.

• Treat paths and metadata as untrusted; escape text previews and block remote resources by default.

• Log errors without dumping private file contents; no default filename/path telemetry.

Architecture decision records

• ADR-001: Tauri + React/TypeScript + Rust; revisit only with measured constraints.

• ADR-002: SQLite local index; FTS5 is optional and opt-in for extracted content.

• ADR-003: modular preview router with explicit support/fallback status.

• ADR-004: local-first privacy and rebuildable cache/index.

References

• Tauri v2 project creation: https://v2.tauri.app/start/

• Tauri filesystem plugin: https://v2.tauri.app/plugin/file-system/

• React + TypeScript: https://react.dev/learn/typescript

• Vite guide: https://vite.dev/guide/

• SQLite FTS5: https://www.sqlite.org/fts5.html

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.